JavaScript is disabled on your browser.

Class CharsetDecoder

java.lang.Object
    java.nio.charset.CharsetDecoder

```
public abstract class CharsetDecoder
extends Object
```

An engine that can transform a sequence of bytes in a specific charset into a sequence of sixteen-bit Unicode characters.

The input byte sequence is provided in a byte buffer or a series of such buffers. The output character sequence is written to a character buffer or a series of such buffers. A decoder should always be used by making the following sequence of method invocations, hereinafter referred to as a *decoding operation*:

1. Reset the decoder via the `reset` method, unless it has not been used before;
2. Invoke the `decode` method zero or more times, as long as additional input may be available, passing `false` for the `endOfInput` argument and filling the input buffer and flushing the output buffer between invocations;
3. Invoke the `decode` method one final time, passing `true` for the `endOfInput` argument; and then
4. Invoke the `flush` method so that the decoder can flush any internal state to the output buffer.

Each invocation of the `decode` method will decode as many bytes as possible from the input buffer, writing the resulting characters to the output buffer. The `decode` method returns when more input is required, when there is not enough room in the output buffer, or when a decoding error has occurred. In each case a `CoderResult` object is returned to describe the reason for termination. An invoker can examine this object and fill the input buffer, flush the output buffer, or attempt to recover from a decoding error, as appropriate, and try again.

There are two general types of decoding errors. If the input byte sequence is not legal for this charset then the input is considered *malformed*. If the input byte sequence is legal but cannot be mapped to a valid Unicode character then an *unmappable character* has been encountered.

How a decoding error is handled depends upon the action requested for that type of error, which is described by an instance of the `CodingErrorAction` class. The possible error actions are to ignore the erroneous input, report the error to the invoker via the returned `CoderResult` object, or replace the erroneous input with the current value of the replacement string. The replacement has the initial value `"\uFFFD"`; its value may be changed via the `replaceWith` method.

The default action for malformed-input and unmappable-character errors is to report them. The malformed-input error action may be changed via the `onMalformedInput` method; the unmappable-character action may be changed via the `onUnmappableCharacter` method.

This class is designed to handle many of the details of the decoding process, including the implementation of error actions. A decoder for a specific charset, which is a concrete subclass of this class, need only implement the abstract `decodeLoop` method, which encapsulates the basic decoding loop. A subclass that maintains internal state should, additionally, override the `implFlush` and `implReset` methods.

Instances of this class are not safe for use by multiple concurrent threads.

Since:

`1.4`

See Also:

`ByteBuffer`, `CharBuffer`, `Charset`, `CharsetEncoder`

Constructor Summary

Constructors

Modifier	Constructor	Description
`protected`	`CharsetDecoder(Charset cs, float averageCharsPerByte, float maxCharsPerByte)`	Initializes a new decoder.

Method Summary

Concrete Methods

Modifier and Type	Method
`float`	`averageCharsPerByte()`

Charset	**charset**()
CharBuffer	**decode**(**ByteBuffer** in)
CoderResult	**decode**(**ByteBuffer** in, **CharBuffer** out, boolean endOfInput)
protected abstract **CoderResult**	**decodeLoop**(**ByteBuffer** in, **CharBuffer** out)
Charset	**detectedCharset**()
CoderResult	**flush**(**CharBuffer** out)
protected **CoderResult**	**implFlush**(**CharBuffer** out)
protected void	**implOnMalformedInput**(**CodingErrorAction** newAction)

protected void	**implOnUnmappableCharacter**(**CodingErrorAction** newAc
protected void	**implReplaceWith**(**String** newReplacement)
protected void	**implReset**()
boolean	**isAutoDetecting**()
boolean	**isCharsetDetected**()
CodingErrorAction	**malformedInputAction**()
float	**maxCharsPerByte**()
CharsetDecoder	**onMalformedInput**(**CodingErrorAction** newAction)

CharsetDecoder	onUnmappableCharacter(CodingErrorAction newAction
String	replacement()
CharsetDecoder	replaceWith(String newReplacement)
CharsetDecoder	reset()
CodingErrorAction	unmappableCharacterAction()

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

CharsetDecoder

```
protected CharsetDecoder(Charset cs,
                         float averageCharsPerByte,
                         float maxCharsPerByte)
```

Initializes a new decoder. The new decoder will have the given chars-per-byte values and its replacement will be the string `"\uFFFD"`.

Parameters:

`cs` - The charset that created this decoder

averageCharsPerByte - A positive float value indicating the expected number of characters that will be produced for each input byte

maxCharsPerByte - A positive float value indicating the maximum number of characters that will be produced for each input byte

Throws:

IllegalArgumentException - If the preconditions on the parameters do not hold

Method Detail

charset

public final Charset charset()

Returns the charset that created this decoder.

Returns:

This decoder's charset

replacement

public final String replacement()

Returns this decoder's replacement value.

Returns:

This decoder's current replacement, which is never null and is never empty

replaceWith

public final CharsetDecoder replaceWith(String newReplacement)

Changes this decoder's replacement value.

This method invokes the implReplaceWith method, passing the new replacement, after checking that the new replacement is acceptable.

Parameters:

newReplacement - The replacement value The new replacement; must not be null and must have non-zero length

Returns:

This decoder

Throws:

`IllegalArgumentException` - If the preconditions on the parameter do not hold

implReplaceWith

```
protected void implReplaceWith(String newReplacement)
```

Reports a change to this decoder's replacement value.

The default implementation of this method does nothing. This method should be overridden by decoders that require notification of changes to the replacement.

Parameters:

`newReplacement` - The replacement value

malformedInputAction

```
public CodingErrorAction malformedInputAction()
```

Returns this decoder's current action for malformed-input errors.

Returns:

The current malformed-input action, which is never null

onMalformedInput

```
public final CharsetDecoder onMalformedInput(CodingErrorAction newAction)
```

Changes this decoder's action for malformed-input errors.

This method invokes the `implOnMalformedInput` method, passing the new action.

Parameters:

`newAction` - The new action; must not be null

Returns:

This decoder

Throws:

`IllegalArgumentException` - If the precondition on the parameter does not hold

implOnMalformedInput

```
protected void implOnMalformedInput(CodingErrorAction newAction)
```

Reports a change to this decoder's malformed-input action.

The default implementation of this method does nothing. This method should be overridden by decoders that require notification of changes to the malformed-input action.

Parameters:

`newAction - The new action`

unmappableCharacterAction

```
public CodingErrorAction unmappableCharacterAction()
```

Returns this decoder's current action for unmappable-character errors.

Returns:

`The current unmappable-character action, which is never null`

onUnmappableCharacter

```
public final CharsetDecoder onUnmappableCharacter(CodingErrorAction newAction)
```

Changes this decoder's action for unmappable-character errors.

This method invokes the `implOnUnmappableCharacter` method, passing the new action.

Parameters:

`newAction - The new action; must not be null`

Returns:

`This decoder`

Throws:

`IllegalArgumentException - If the precondition on the parameter does not hold`

implOnUnmappableCharacter

```
protected void implOnUnmappableCharacter(CodingErrorAction newAction)
```

Reports a change to this decoder's unmappable-character action.

The default implementation of this method does nothing. This method should be overridden by decoders that require notification of changes to the unmappable-character action.

Parameters:

`newAction - The new action`

averageCharsPerByte

```
public final float averageCharsPerByte()
```

Returns the average number of characters that will be produced for each byte of input. This heuristic value may be used to estimate the size of the output buffer required for a given input sequence.

Returns:

`The average number of characters produced per byte of input`

maxCharsPerByte

```
public final float maxCharsPerByte()
```

Returns the maximum number of characters that will be produced for each byte of input. This value may be used to compute the worst-case size of the output buffer required for a given input sequence.

Returns:

`The maximum number of characters that will be produced per byte of input`

decode

```
public final CoderResult decode(ByteBuffer in,
                                CharBuffer out,
                                boolean endOfInput)
```

Decodes as many bytes as possible from the given input buffer, writing the results to the given output buffer.

The buffers are read from, and written to, starting at their current positions. At most `in.remaining()` bytes will be read and at most `out.remaining()` characters will be written. The buffers' positions will be advanced to reflect the bytes read and the characters written, but their marks and limits will not be modified.

In addition to reading bytes from the input buffer and writing characters to the output buffer, this method returns a `CoderResult` object to describe its reason for termination:

- `CoderResult.UNDERFLOW` indicates that as much of the input buffer as possible has been decoded. If there is no further input then the invoker can proceed to the next step of the decoding operation. Otherwise this method should be invoked again with further input.
- `CoderResult.OVERFLOW` indicates that there is insufficient space in the output buffer to decode any more bytes. This method should be invoked again with an output buffer that has more remaining characters. This is typically done by draining any decoded characters from the output buffer.
- A malformed-input result indicates that a malformed-input error has been detected. The malformed bytes begin at the input buffer's (possibly incremented) position; the number of malformed bytes may be determined by invoking the result object's `length` method. This case applies only if the malformed action of this decoder is `CodingErrorAction.REPORT`; otherwise the malformed input will be ignored or replaced, as requested.
- An unmappable-character result indicates that an unmappable-character error has been detected. The bytes that decode the unmappable character begin at the input buffer's (possibly incremented) position; the number of such bytes may be determined by invoking the result object's `length` method. This case applies only if the unmappable action of this decoder is `CodingErrorAction.REPORT`; otherwise the unmappable character will be ignored or replaced, as requested.

In any case, if this method is to be reinvoked in the same decoding operation then care should be taken to preserve any bytes remaining in the input buffer so that they are available to the next invocation.

The `endOfInput` parameter advises this method as to whether the invoker can provide further input beyond that contained in the given input buffer. If there is a possibility of providing additional input then the invoker should pass `false` for this parameter; if there is no possibility of providing further input then the invoker should pass `true`. It is not erroneous, and in fact it is quite common, to pass `false` in one invocation and later discover that no further input was actually available. It is critical, however, that the final invocation of this method in a sequence of invocations always pass `true` so that any remaining undecoded input will be treated as being malformed.

This method works by invoking the `decodeLoop` method, interpreting its results, handling error conditions, and reinvoking it as necessary.

Parameters:

`in - The input byte buffer`

out - The output character buffer

endOfInput - true if, and only if, the invoker can provide no additional input bytes beyond those in the given buffer

Returns:

A coder-result object describing the reason for termination

Throws:

IllegalStateException - If a decoding operation is already in progress and the previous step was an invocation neither of the reset method, nor of this method with a value of false for the endOfInput parameter, nor of this method with a value of true for the endOfInput parameter but a return value indicating an incomplete decoding operation

CoderMalfunctionError - If an invocation of the decodeLoop method threw an unexpected exception

flush

public final CoderResult flush(CharBuffer out)

Flushes this decoder.

Some decoders maintain internal state and may need to write some final characters to the output buffer once the overall input sequence has been read.

Any additional output is written to the output buffer beginning at its current position. At most out.remaining() characters will be written. The buffer's position will be advanced appropriately, but its mark and limit will not be modified.

If this method completes successfully then it returns CoderResult.UNDERFLOW. If there is insufficient room in the output buffer then it returns CoderResult.OVERFLOW. If this happens then this method must be invoked again, with an output buffer that has more room, in order to complete the current decoding operation.

If this decoder has already been flushed then invoking this method has no effect.

This method invokes the implFlush method to perform the actual flushing operation.

Parameters:

out - The output character buffer

Returns:

A coder-result object, either CoderResult.UNDERFLOW or CoderResult.OVERFLOW

Throws:

`IllegalStateException` - If the previous step of the current decoding operation was an invocation neither of the `flush` method nor of the three-argument `decode` method with a value of true for the endOfInput parameter

implFlush

```
protected CoderResult implFlush(CharBuffer out)
```

Flushes this decoder.

The default implementation of this method does nothing, and always returns `CoderResult.UNDERFLOW`. This method should be overridden by decoders that may need to write final characters to the output buffer once the entire input sequence has been read.

Parameters:

`out` - The output character buffer

Returns:

A coder-result object, either `CoderResult.UNDERFLOW` or `CoderResult.OVERFLOW`

reset

```
public final CharsetDecoder reset()
```

Resets this decoder, clearing any internal state.

This method resets charset-independent state and also invokes the `implReset` method in order to perform any charset-specific reset actions.

Returns:

This decoder

implReset

```
protected void implReset()
```

Resets this decoder, clearing any charset-specific internal state.

The default implementation of this method does nothing. This method should be overridden by decoders that maintain internal state.

decodeLoop

```
protected abstract CoderResult decodeLoop(ByteBuffer in,
                                          CharBuffer out)
```

Decodes one or more bytes into one or more characters.

This method encapsulates the basic decoding loop, decoding as many bytes as possible until it either runs out of input, runs out of room in the output buffer, or encounters a decoding error. This method is invoked by the `decode` method, which handles result interpretation and error recovery.

The buffers are read from, and written to, starting at their current positions. At most `in.remaining()` bytes will be read, and at most `out.remaining()` characters will be written. The buffers' positions will be advanced to reflect the bytes read and the characters written, but their marks and limits will not be modified.

This method returns a `CoderResult` object to describe its reason for termination, in the same manner as the `decode` method. Most implementations of this method will handle decoding errors by returning an appropriate result object for interpretation by the `decode` method. An optimized implementation may instead examine the relevant error action and implement that action itself.

An implementation of this method may perform arbitrary lookahead by returning `CoderResult.UNDERFLOW` until it receives sufficient input.

Parameters:

`in - The input byte buffer`

`out - The output character buffer`

Returns:

`A coder-result object describing the reason for termination`

decode

```
public final CharBuffer decode(ByteBuffer in)
                        throws CharacterCodingException
```

Convenience method that decodes the remaining content of a single input byte buffer into a newly-allocated character buffer.

This method implements an entire decoding operation; that is, it resets this decoder, then it decodes the bytes in the given byte buffer, and finally it flushes this decoder. This method should therefore not be invoked if a decoding operation is already in progress.

Parameters:

in - The input byte buffer

Returns:

A newly-allocated character buffer containing the result of the decoding operation. The buffer's position will be zero and its limit will follow the last character written.

Throws:

IllegalStateException - If a decoding operation is already in progress

MalformedInputException - If the byte sequence starting at the input buffer's current position is not legal for this charset and the current malformed-input action is CodingErrorAction.REPORT

UnmappableCharacterException - If the byte sequence starting at the input buffer's current position cannot be mapped to an equivalent character sequence and the current unmappable-character action is CodingErrorAction.REPORT

CharacterCodingException

isAutoDetecting

```
public boolean isAutoDetecting()
```

Tells whether or not this decoder implements an auto-detecting charset.

The default implementation of this method always returns `false`; it should be overridden by auto-detecting decoders to return `true`.

Returns:

true if, and only if, this decoder implements an auto-detecting charset

isCharsetDetected

```
public boolean isCharsetDetected()
```

Tells whether or not this decoder has yet detected a charset *(optional operation)*.

If this decoder implements an auto-detecting charset then at a single point during a decoding operation this method may start returning `true` to indicate that a specific charset has been detected in the input byte sequence. Once this occurs, the detectedCharset method may be invoked to retrieve the detected charset.

That this method returns `false` does not imply that no bytes have yet been decoded. Some auto-detecting decoders are capable of decoding some, or even all, of an input byte sequence without fixing on a particular charset.

The default implementation of this method always throws an `UnsupportedOperationException`; it should be overridden by auto-detecting decoders to return `true` once the input charset has been determined.

Returns:

`true if, and only if, this decoder has detected a specific charset`

Throws:

`UnsupportedOperationException - If this decoder does not implement an auto-detecting charset`

detectedCharset

```
public Charset detectedCharset()
```

Retrieves the charset that was detected by this decoder *(optional operation)*.

If this decoder implements an auto-detecting charset then this method returns the actual charset once it has been detected. After that point, this method returns the same value for the duration of the current decoding operation. If not enough input bytes have yet been read to determine the actual charset then this method throws an `IllegalStateException`.

The default implementation of this method always throws an `UnsupportedOperationException`; it should be overridden by auto-detecting decoders to return the appropriate value.

Returns:

`The charset detected by this auto-detecting decoder, or null if the charset has not yet been determined`

Throws:

`IllegalStateException - If insufficient bytes have been read to determine a charset`

```
UnsupportedOperationException - If this decoder does not implement
an auto-detecting charset
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.